

LangChain & Agentic AI – From Basics to Advanced demo- 6 Notes

Multiple Messages and Conversation Context

```
import os

from langchain_core.messages import SystemMessage,
HumanMessage, AIMessage

from langchain_openai import ChatOpenAI

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") #function

llm = ChatOpenAI(model="gpt-4o",api_key=OPENAI_API_KEY)

human_message = HumanMessage(content = "Hello, My Name
is Durga")

ai_message = llm.invoke("Hello My Name is Durga")

print(ai_message.content)

ai_message = llm.invoke("Oh wonderful LLM, What is my
Name:")

print(ai_message.content)
```

Context: It is information available to the model that helps it understand current request.

Note:

```
human_message = HumanMessage(content = "Hello, My Name is Durga")
```

```
ai_message = llm.invoke(human_message) #ValueError
```

```
ai_message = llm.invoke([human_message])
```

```
import os
```

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
```

```
from langchain_openai import ChatOpenAI
```

```
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") #function
```

```
llm = ChatOpenAI(model="gpt-4o",api_key=OPENAI_API_KEY)
```

```
human_message1 = HumanMessage(content = "Hello, My Name is Durga")
```

```
ai_message1 = llm.invoke([human_message1])  
print(ai_message1.content)
```

```
human_message2 = HumanMessage(content = "What is my  
name?")
```

```
ai_message2 = llm.invoke([human_message1, ai_message1,  
human_message2])  
print(ai_message2.content)
```

```
import os
```

```
from langchain_core.messages import SystemMessage,  
HumanMessage, AIMessage
```

```
from langchain_openai import ChatOpenAI
```

```
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") #function
```

```
llm = ChatOpenAI(model="gpt-4o",api_key=OPENAI_API_KEY)
```

```
msgs = []
```

```
sys_msg = SystemMessage( content = "You are a helpful  
assistant and answer in simple english and in concise way")
```

```
msgs.append(sys_msg)
```

```
while True:
```

```
    question = input("You: ")
```

```
    if question.lower() == "exit":
```

```
        break
```

```
    msgs.append(HumanMessage(content = question))
```

```
    print('The Number of messages sending:', len(msgs))
```

```
    ai_message = llm.invoke(msgs)
```

```
    msgs.append(ai_message)
```

```
    print('AI:', ai_message.content)
```

Context: It is information available to the model that helps it understand current request.

Context Window: It is the maximum amount of information an LLM can consider at one time while generating response.

Context Window size is measured in terms of tokens.

Model	Context window
-----	-----:
GPT-5.6 Sol	**1.05 million tokens**
GPT-5.6 Terra	**1.05 million tokens**
GPT-5.6 Luna	**1.05 million tokens**
GPT-5.4	1.05 million tokens
GPT-5.4 Mini	400,000 tokens
GPT-5.4 Nano	400,000 tokens
GPT-4.1	1,047,576 tokens (~1M)
GPT-4o	128,000 tokens